

System Overview

This project implements a face verification system that determines whether two face images belong to the same person. The system takes two image paths as input, loads and preprocesses both images, generates embedding vectors using a pretrained face embedding model, computes cosine similarity between the embeddings, applies an operating threshold, and returns a prediction.

The inference pipeline looks like this:

Image A + Image B → deterministic preprocessing → embedding generation → cosine similarity scoring → threshold decision → confidence calculation → CLI output

The command-line interface outputs the left image path, right image path, similarity score, threshold, prediction, confidence, and latency.

Intended Use

This system is intended for educational purposes and to experiment with face verification. It is designed to demonstrate a reproducible machine learning pipeline that includes deterministic data handling, embedding-based inference, thresholding, Docker packaging, and runtime profiling. Appropriate uses include academic evaluation of a face verification pipeline, testing pair-level face verification on LFW image pairs, comparing similarity scores between face embeddings, and studying system behavior under different runtime conditions, as we are doing. This system is not intended for real-world identity verification, including use cases such as law enforcement, border control, hiring, access, surveillance, or for any high-stakes decisions. The system has not been validated for production-level deployment or for use across diverse real-world conditions.

Data Summary

The system uses the Labeled Faces in the Wild, or LFW, dataset from `tensorflow_datasets`, which contains face images collected from real-world settings. In this project, LFW is used to create deterministic image pairs for evaluating whether two images belong to the same identity. The project preserves reproducibility by using deterministic ingestion and pair generation, using a fixed seed and consistent split policy so that the same train, validation, and test pair files can be regenerated across runs.

Important data limitations should be considered when interpreting the system. LFW is not fully representative of all real-world populations and may contain uneven demographic representation.

The images also vary in lighting, pose, resolution, facial expression, background, and image quality. In addition, this project does not include reliable demographic labels, so it does not support a formal subgroup fairness evaluation. Because of these limitations, performance on LFW should be utilized for academic purposes rather than real deployment settings.

Operating Threshold

The final operating threshold used by the CLI is: 0.30612244897959173. If $\text{score} \geq \text{threshold}$, $\text{prediction} = 1$, meaning the same identity. If $\text{score} < \text{threshold}$, $\text{prediction} = 0$, meaning different identity. This threshold is used consistently in the final inference script.

The system defines confidence as the absolute distance between the similarity score and the operating threshold. It represents how far the score is from the decision boundary. A score far away from the threshold is treated as a more confident prediction, while a score close to the threshold is treated as less certain. For example, if the score is 0.410 and the threshold is 0.306, then the confidence is 0.104. This means the model predicted the same identity with a margin above the threshold.

Key Metrics

The final CLI uses the embedding-based inference path with cosine similarity and an operating threshold of 0.30612244897959173. For each image pair, the system reports the similarity score, binary prediction, confidence, and latency. The confidence value is defined as the absolute distance between the similarity score and the operating threshold.

The earlier Milestone 2 evaluation results are included as baseline evaluation context. The raw pixel baseline achieved a balanced accuracy of 0.495, while the best normalization-based run achieved a balanced accuracy of 0.5875. These values were produced during the evaluation stage and should be interpreted as course-project metrics, not production-level performance guarantees.

Metric	Value
Operating threshold	.30612244897959173
Scoring method	Cosine similarity
Embedding model	InceptionResnetV1 pretrained on VGGFace2

Inference outputs	score, prediction, confidence, latency
Confidence definition	absolute distance between score and threshold
Milestone 2 baseline balanced accuracy	.495
Milestone 2 best normalization balanced accuracy	.5875

Failure Modes and Limitations

The system may produce unreliable predictions under several conditions. These include poor lighting or extreme shadows, blurry or low-resolution images, strong pose variation, occlusions such as sunglasses, masks, hats, or hands, cropped faces or faces not centered in the image, large age differences between images of the same person, visually similar different-identity pairs, and images outside the distribution of LFW.

The confidence score can also be misleading because it only measures distance from the threshold. It does not measure the true probability that the prediction is correct.

Fairness-Related Risks and Misuse Concerns

Face verification systems can create fairness and misuse risks, especially when used in real-world identity decisions. This project does not include reliable demographic metadata, so it does not make claims about equal performance across demographic groups. Potential fairness-related risks include uneven performance across groups due to dataset imbalance, lower reliability on image conditions underrepresented in the dataset, higher error rates for faces affected by lighting, pose, occlusion, or image quality, and harmful use in surveillance or high-stakes identity decisions. This system should not be used to make decisions that affect legal rights, access to services, employment, financial access, public safety, or personal freedom.

Operational Constraints

The system has several practical constraints. It is designed for pair-level inference, not large-scale production search. The CLI expects two valid image paths or LFW-relative paths. Runtime depends on CPU performance, memory availability, and whether Docker is used. The system currently relies on local environment setup or Docker packaging. The model download and dependency installation can be large because TensorFlow and PyTorch are both used. The Dockerized system is intended for reproducibility, not optimized production deployment.

A CPU baseline profile should be included separately in the profiling report. The profiling report should measure preprocessing latency, embedding latency, scoring latency, and end-to-end latency.

Reproducibility Pointer

The final release can be reproduced using the repository README and reproducibility checklist.

The final tag is v1.0-final. The tag should point to the exact commit containing the final README, System Card, profiling report, reproducibility checklist, Dockerfile, and inference code.